

REPUBLIQUE DU SENEGAL



Un Peuple – Un But – Une Foi

MINISTERE DE L'ENSEIGNEMENT SUPERIEUR, DE LA RECHERCHE ET DE L'INNOVATION

ECOLE POLYTECHNIQUE DE THIÈS



DÉPARTEMENT INFORMATIQUE ET TÉLÉCOMMUNICATIONS

Master 1 en ingénierie logicielle et intelligence artificielle

Data Engineering

Projet 3 : Campagnes marketing

Présenté par :

Aliou DRAME

Khady KA

Seydina Mouhamed Lybass POUYE

Ndongo MBATH

Encadreur :

M. SOW

Année académique : **2025/2026**

1 .Contexte et objectifs

Une entreprise souhaite améliorer le retour sur investissement (ROI) de ses campagnes marketing en fusionnant des données issues de multiples systèmes hétérogènes : logs de navigation web, CRM et plateformes publicitaires. L'objectif de ce projet est de construire un pipeline de data engineering complet permettant d'obtenir une vision globale et fiable des performances de campagne.

Le projet couvre l'ensemble de la chaîne de valeur data :

- Ingestion multi-source de données hétérogènes
- Nettoyage, transformation et jointure des données (ETL)
- Calcul de KPIs marketing (taux de conversion, coût par acquisition, portée...)
- Orchestration automatisée des traitements
- Visualisation des performances via un dashboard interactif

2 .Architecture générale

L'architecture retenue est une architecture Lakehouse simplifiée, dite « médaille » (bronze / silver / gold), largement utilisée en data engineering pour organiser un data lake par niveau croissant de qualité et d'agrégation des données

Couche	Contenu	Format	Rôle
Bronze	Données brutes, telles que déposées par les sources	CSV	Traçabilité et rejouabilité (on peut toujours revenir à la donnée source)
Silver	Données nettoyées, normalisées et jointes	Parquet	Table de faits unique « sessions_enriched » exploitable pour l'analyse
Gold	KPIs agrégés par campagne	Parquet	Données prêtes à consommer par le dashboard, sans recalcul côté BI

Le flux de données suit le schéma suivant :

- CRM (csv) + Publicités (csv) + Web logs (csv) → Airflow DAG
- Airflow → MinIO/bronze (ingestion brute)
- MinIO/bronze → nettoyage + jointure → MinIO/silver
- MinIO/silver → calcul des KPIs → MinIO/gold
- MinIO/gold → Dashboard Streamlit

Ce découpage en zones permet d'isoler les responsabilités :

- La zone bronze garantit qu'aucune donnée source n'est perdue ou modifiée en place.
- La zone silver centralise la logique de nettoyage/jointure en un seul endroit testable .
- la zone gold isole le calcul métier (KPIs) du reste du pipeline, ce qui facilite l'ajout de nouveaux indicateurs sans toucher aux étapes précédentes.

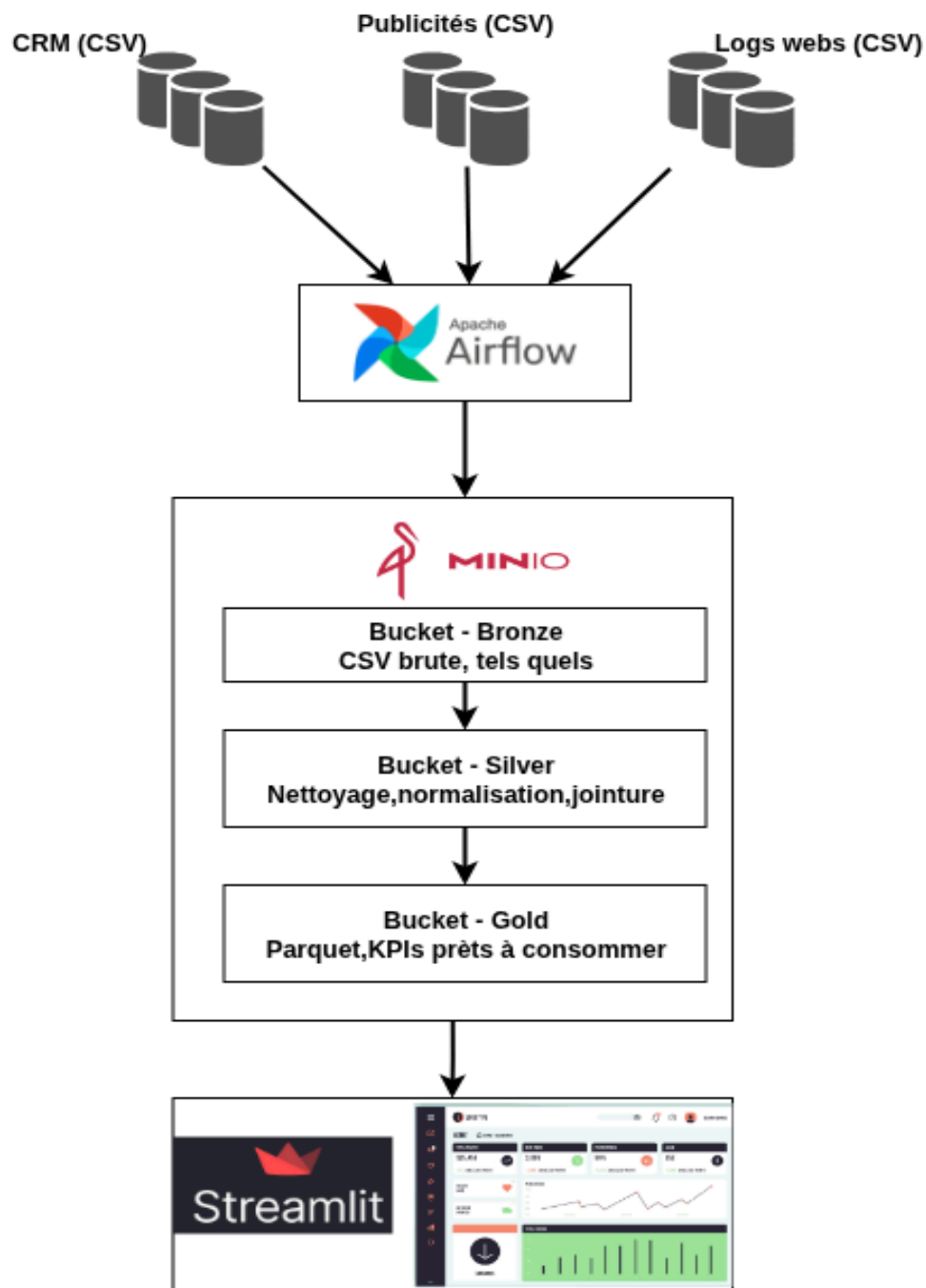


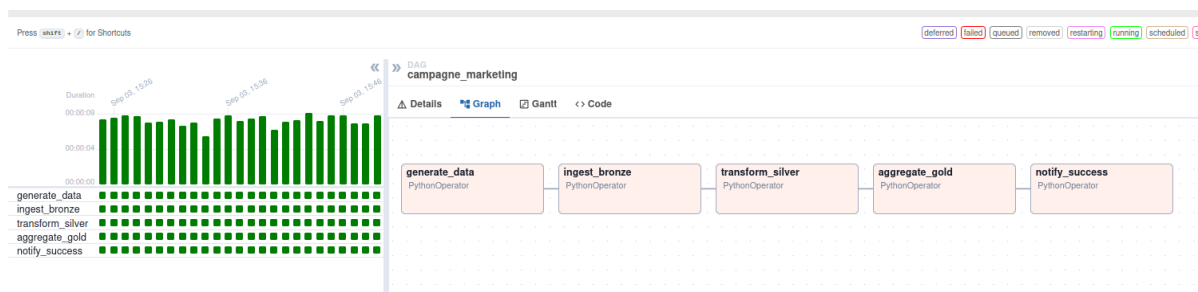
figure 1: Architecture et pepilne

3. Choix technologiques et justification

3.1 Orchestration Apache Airflow

Apache Airflow a été retenu pour orchestrer le pipeline ETL plutôt qu'une simple planification par cron ou un outil plus léger.

Critère	Apache Airflow (retenu)	Cron + scripts
Visibilité de l'exécution	Interface web complète (statut, logs, durée par tâche)	Aucune, sauf logs manuels
Gestion des dépendances entre tâches	Native (DAG explicite)	Manuelle, fragile
Reprise sur erreur / retries	Native, configurable par tâche	À coder soi-même

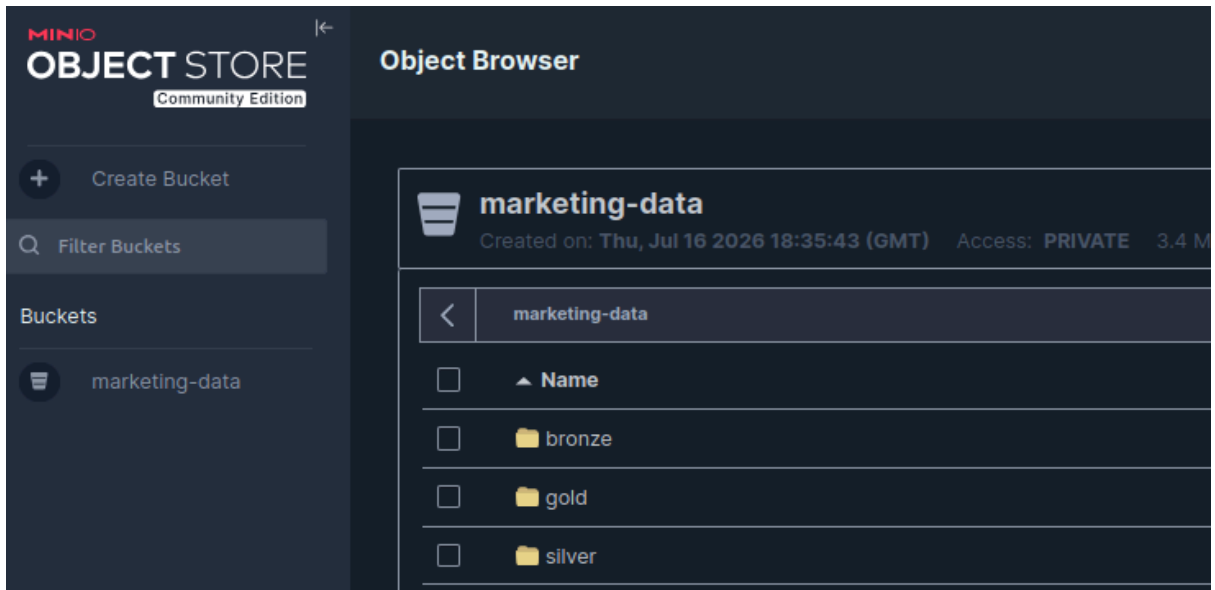


capture-airflow : le dag airflow du projet campagne marketing

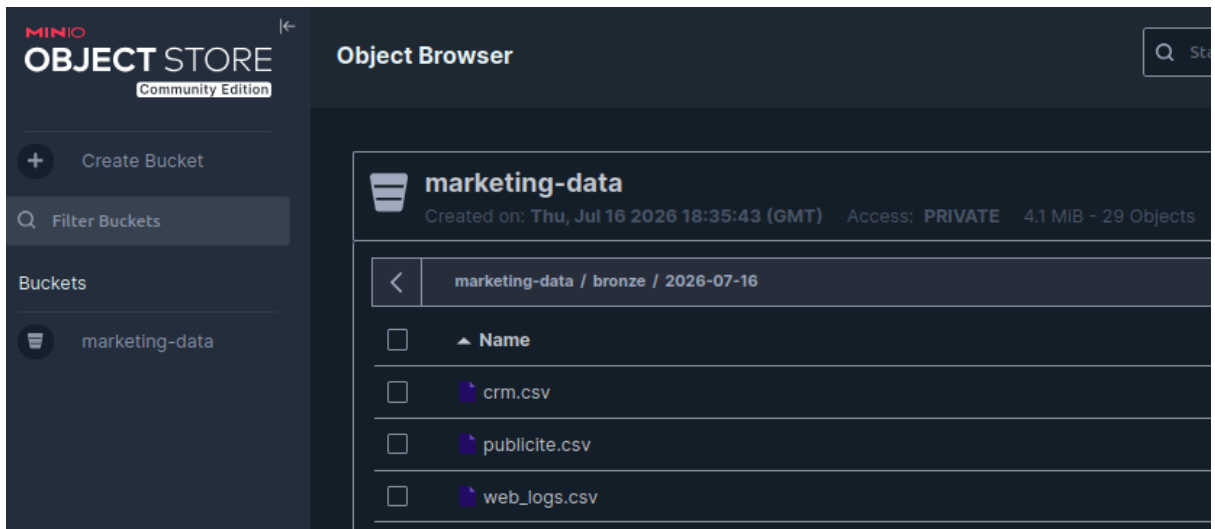
3.2 Stockage MinIO (data lake S3-compatible)

MinIO a été choisi comme couche de stockage plutôt qu'une base de données relationnelle classique ou de simples fichiers locaux.

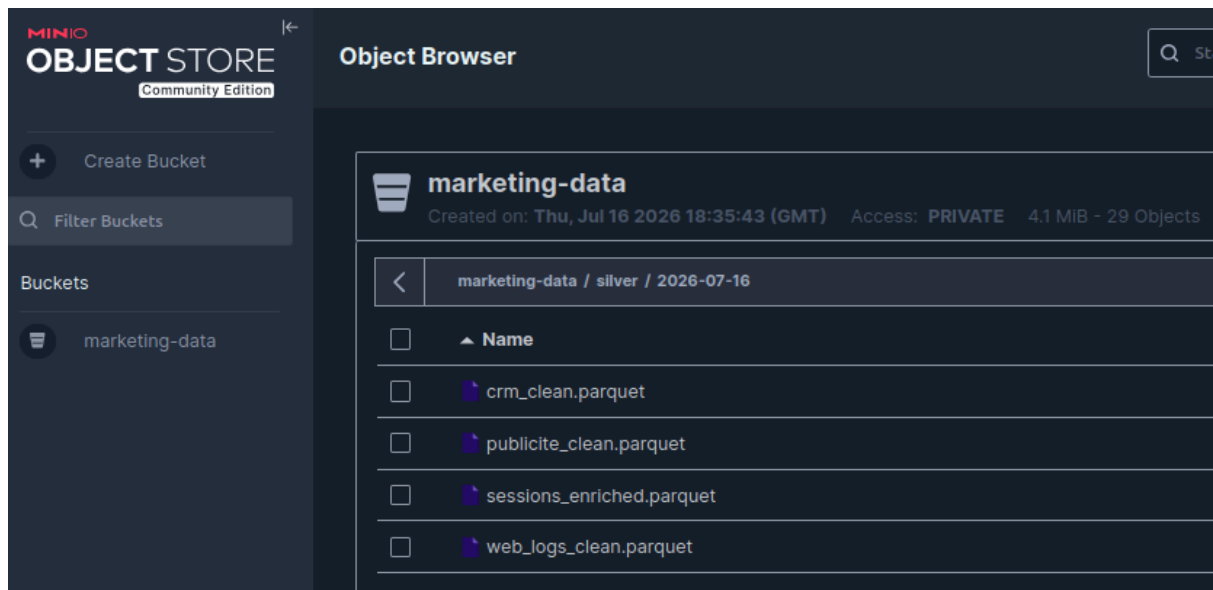
- MinIO expose une API compatible S3, ce qui rapproche l'architecture d'un environnement cloud réel (AWS S3, GCS) sans dépendre d'un fournisseur cloud payant.
- Le stockage objet convient naturellement à une architecture médaille en zones (bronze/silver/gold), organisées comme des préfixes de clés plutôt que comme des schémas de base de données.
- Le format Parquet (colonnes, compressé) est directement exploitable depuis MinIO par Pandas, sans étape d'import supplémentaire.
- Contrairement à une base relationnelle, MinIO n'impose pas de schéma figé à l'écriture, ce qui facilite l'évolution des sources de données.



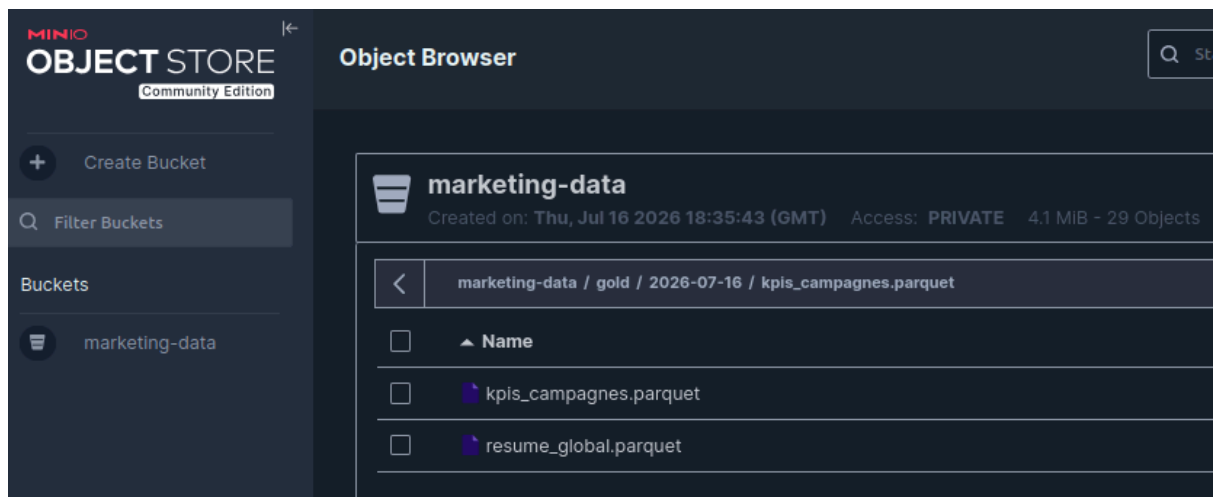
capture-minio 1 : les dossiers bronze,silver et gold dans minio



capture-minio 2 : les fichiers csv brute de la couche bronze



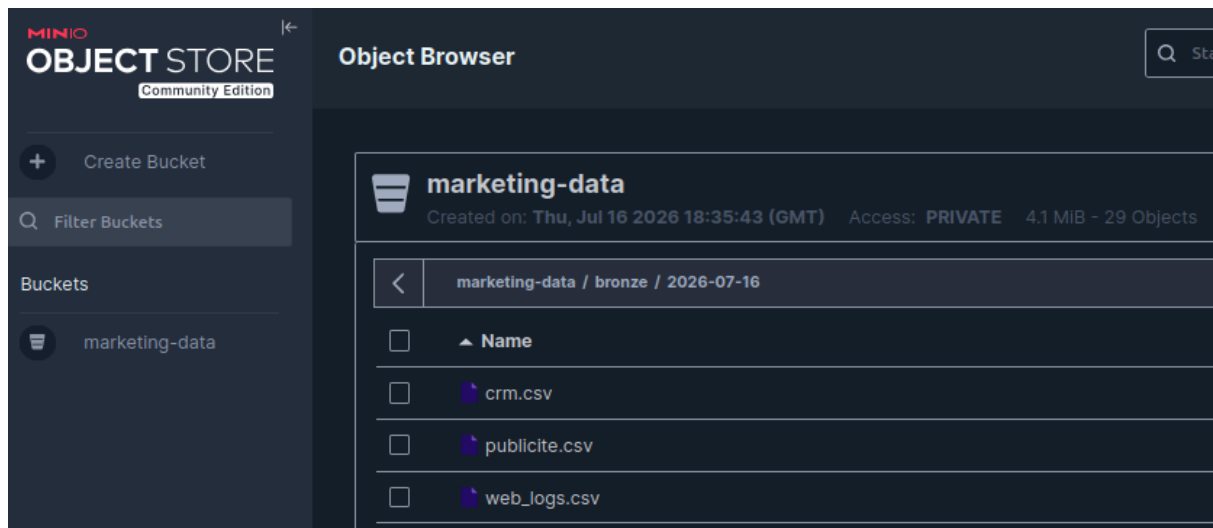
capture-minio 3 : les fichiers parquet nettoyés de la couche silver



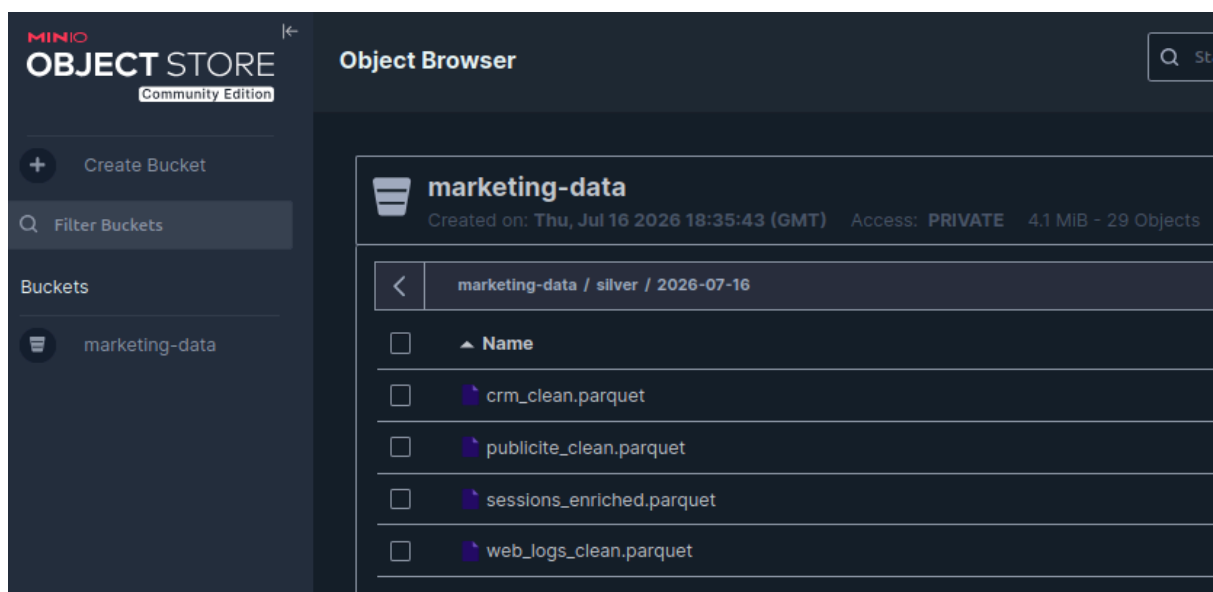
capture-minio 4 : les fichiers parquet de la couche gold

3.3 Format de données CSV en entrée, Parquet en interne

Les données sources (bronze) restent en CSV, format universel et lisible par toute source réelle (export CRM, export plateforme publicitaire). Dès la zone silver, les données sont converties en Parquet : format colonnaire, typé, compressé, nettement plus performant que le CSV pour les lectures analytiques répétées (le dashboard Streamlit relit les données à chaque rafraîchissement).



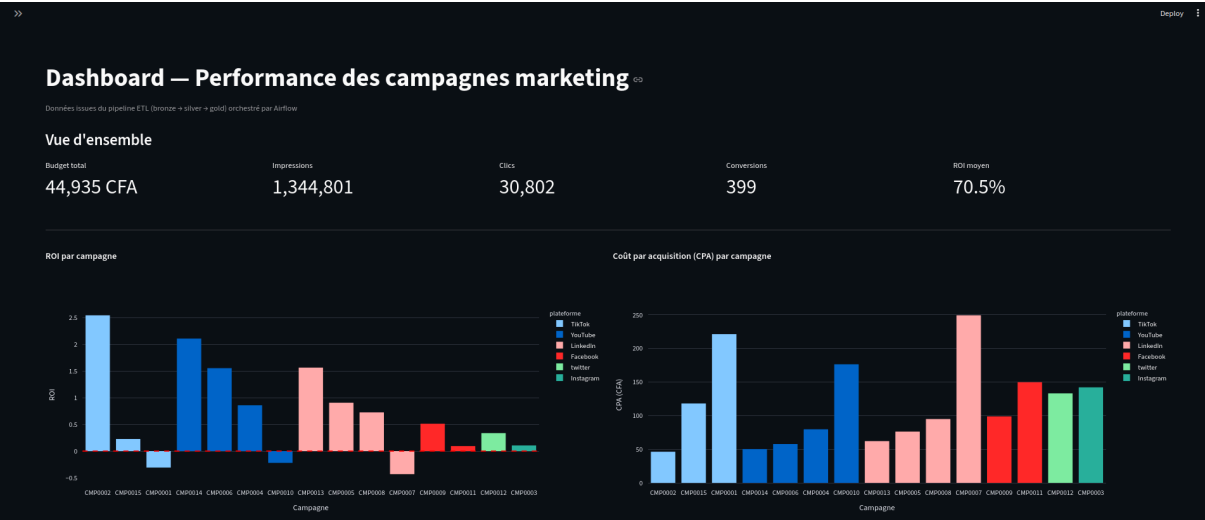
capture-minio 5 : les fichiers csv brute de la couche bronze



capture-minio 6 : les fichiers parquet nettoyés de la couche silver

3.4 Visualisation Streamlit

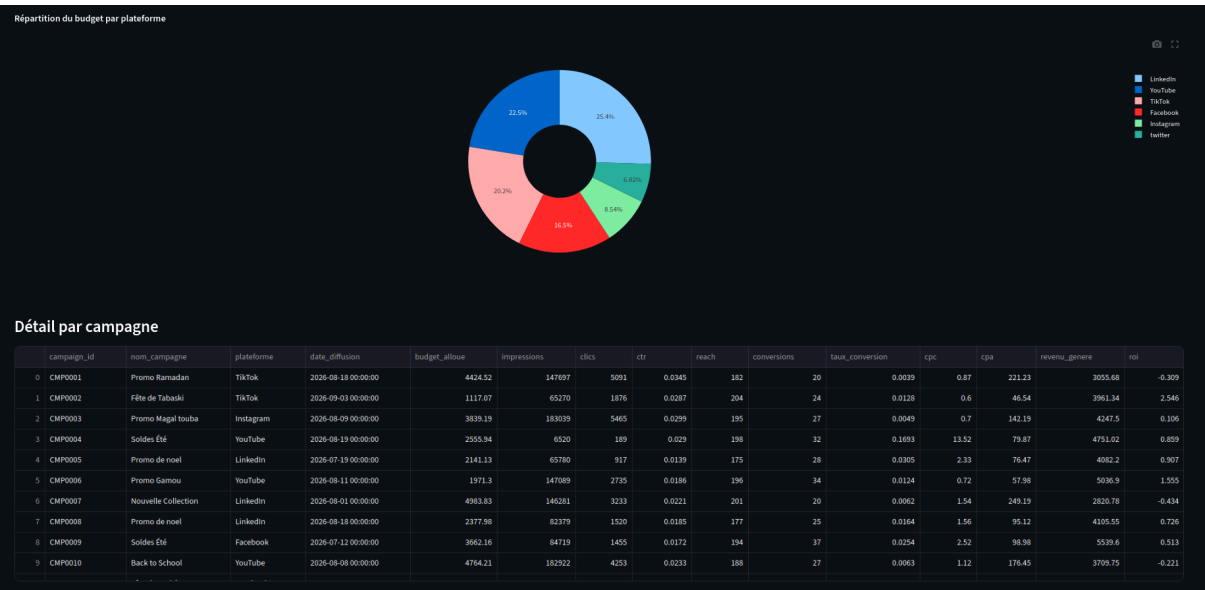
Streamlit permet de construire un dashboard interactif entièrement en Python, sans changer d'écosystème par rapport au reste du pipeline . Comparé à des outils BI comme Power BI ou Metabase, Streamlit offre moins de fonctionnalités « clé en main » mais une flexibilité totale sur la logique métier et une intégration immédiate avec MinIO via le même client S3 que les scripts ETL.



capture-streamlit 1 : vue d'ensemble des KPI



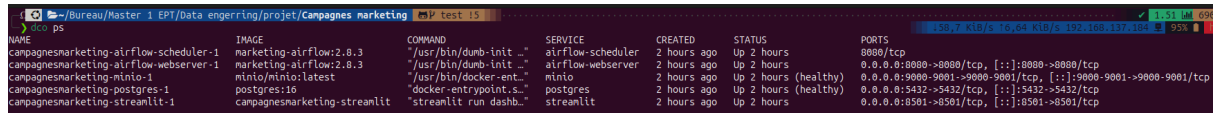
capture-streamlit 2 : ROI par campagne et Coût par acquisition (CPA) par campagne



capture-streamlit 3 : Répartition du budget par plateforme et Détail par campagne

3.5 Conteneurisation Docker Compose

L'ensemble des services (Postgres, MinIO, Airflow, Streamlit) est orchestré via Docker Compose, ce qui garantit un environnement reproductible sur n'importe quelle machine et évite les problèmes de dépendances Python conflictuelles entre composants.



NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
campagnesmarketing-airflow-scheduler-1	marketing-airflow:2.8.3	"/usr/bin/dumb-init -"	airflow-scheduler	2 hours ago	Up 2 hours	8080/tcp
campagnesmarketing-airflow-webserver-1	marketing-airflow:2.8.3	"/usr/bin/dumb-init -"	airflow-webserver	2 hours ago	Up 2 hours	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
campagnesmarketing-minio-1	minio/minio:latest	"/usr/bin/docker-ent..."	minio	2 hours ago	Up 2 hours (healthy)	0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp
campagnesmarketing-postgres-1	postgres:16	"docker-entrypoint.s..."	postgres	2 hours ago	Up 2 hours (healthy)	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
campagnesmarketing-streamlit-1	campagnesmarketing-streamlit	"streamlit run dashb..."	streamlit	2 hours ago	Up 2 hours	0.0.0.0:8501->8501/tcp, [::]:8501->8501/tcp

capture-conteneurs : les conteneurs du projet

3.6 Génération de données Faker

Les 3 jeux de données fictifs (CRM, publicités, logs web) sont générés avec la librairie Faker (locale française), avec une graine aléatoire fixe pour garantir la reproductibilité des tests. Les données sont volontairement liées entre elles (mêmes identifiants client/campagne partagés entre les tables) afin de simuler un cas réaliste de fusion de systèmes

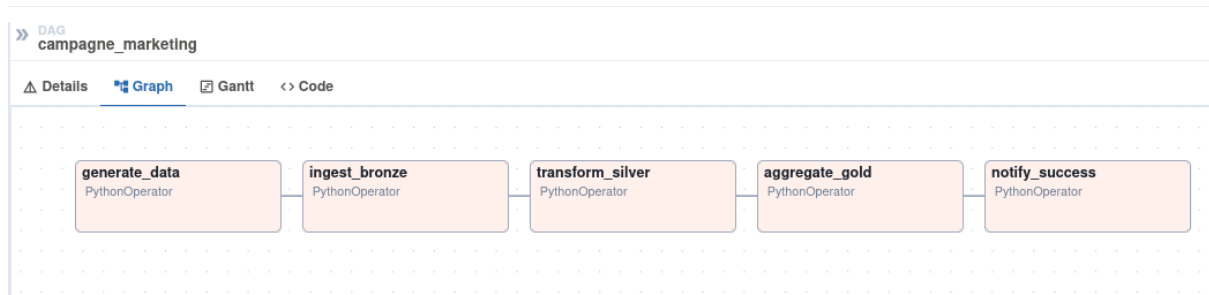
4. KPIs calculés (couche gold)

KPI	Formule	Signification métier
CTR	clics / impressions	Taux de clic sur la publicité
Taux de conversion	conversions / clics	Efficacité du tunnel de conversion
Reach	nombre d'utilisateurs uniques touchés	Portée réelle de la campagne
CPC	budget / clics	Coût par clic
CPA	budget / conversions	Coût par acquisition d'un client
ROI	(revenu généré – budget) / budget	Retour sur investissement

5. Orchestration du pipeline

Le DAG Airflow marketing_campaign_etl s'exécute quotidiennement (schedule_interval=@daily par exemple) et enchaîne 5 tâches séquentielles :

- generate_data : simule/collecte les données du jour
- ingest_bronze : dépose les CSV bruts dans MinIO
- transform_silver : nettoie, normalise et joint les 3 sources
- aggregate_gold : calcule les KPIs par campagne
- notify_success : journalise la fin d'exécution



capture-airflow : le dag airflow du projet campagne marketing

Chaque tâche est un script Python indépendant (dossier etl/), ce qui permet de les tester unitairement en dehors d'Airflow et de les rejouer individuellement en cas d'échec (mécanisme de retries natif d'Airflow, configuré à 2 tentatives avec 1 minutes de délai).

6. Bonnes pratiques et sécurité

- Les identifiants (MinIO, Postgres, Airflow) sont injectés via des variables d'environnement Docker Compose et un fichier .env, jamais codés en dur dans les scripts Python.
- Les scripts ETL sont idempotents par date d'exécution (run_date) : rejouer le pipeline pour une même date écrase proprement les données de cette date sans dupliquer.
- La table gold « latest » est toujours synchronisée avec le run le plus récent, ce qui simplifie la lecture côté dashboard tout en conservant l'historique par date dans les zones bronze/silver/gold.

Conclusion

Ce projet met en œuvre une architecture de data engineering complète et représentative des pratiques d'entreprise : ingestion multi-source, architecture medallion sur un data lake S3-compatible, orchestration par Airflow et restitution via un dashboard interactif. L'ensemble est conteneurisé et reproductible, ce qui permet de le déployer et de l'exécuter de bout en bout sur n'importe quelle machine disposant de Docker.

Lien GitHub : <https://github.com/ndongombathie/campagne-de-marketing>